

Types as expressions

First-class types in D

Dennis Korpel

It keeps coming up

“ What about allowing

```
alias T = cond ? U : V;
```

as a shorthand for

```
static if (cond) alias T = U; else alias T = V;
```

?

”

Per Nordlöw — April 15, 2026, "Type-Level Ternary Expressions"

<https://forum.dlang.org/thread/yyjjebcbkmmmtmvleywkr@forum.dlang.org>

And it kept coming up

“ The algorithms follow the following pattern consistently:

1. **Reify** the compile-time parameters into values
 2. Carry processing on these values with the usual algorithms
 3. If needed, **dereify** back the results into compile-time parameters
- ”

Andrei Alexandrescu — September 26, 2020, "Reimplementing the bulk of std.meta iteratively"

[https://forum.dlang.org/thread/rknpkj\\$d7f\\$1@digitalmars.com](https://forum.dlang.org/thread/rknpkj$d7f$1@digitalmars.com)

What Andrei used

1. Convert a type to `string` with `.stringof`
2. Pass string **values** around
3. Convert back with `mixin()`

What you can do today

1. Convert a type to `string` with `.mangleof`
2. Pass string **values** around
3. Convert back with `__traits(toType, str)`

Example

```
string unsignedOfSize(size_t n)
{
    if (n == 4)
        return uint.mangleof;
    if (n == 2)
        return ushort.mangleof;
    if (n == 1)
        return ubyte.mangleof;
    assert(0);
}

__traits(toType, unsignedOfSize(4)) myInteger;
```

Works today

```
alias ToType(string val) = __traits(toType, val);  
  
ToType!(unsignedOfSize(4)) myInteger;
```

Ordinary functions, ordinary control flow, ordinary data structures.

But we are smuggling types through `string`.

Idea: a type in an expression has type `type_t`

```
type_t myInt = int;  
type_t[3] floats = [float, double, real];
```

No mangling, no `mixin`, no `AliasSeq`.

Works: static array parsing

```
auto foo(alias X, alias Y)()
{
    return X[Y].init;
}
```

```
alias f0 = foo!(int, 3); // int[3].init = [0, 0, 0]
char f1 = foo!("abc", 3); // "abc"[3].init = char.init = 0xFF
```

The grammar already lets `x[Y]` mean both.

Works: **switch**

```
type_t unsignedOf(type_t a)
{
    switch (a)
    {
        case int:  return uint;
        case long: return ulong;
        default:
            assert(0);
    }
}
```

Works: associative arrays

```
enum toUnsigned = [int: uint, short: ushort, byte: ubyte, long: ulong];  
  
static assert(toUnsigned[int] == uint);  
static assert(toUnsigned[short] == ushort);
```

Requires a parser change

```
type_t toPtr(type_t T)
{
    return T*;          // `T*` starts a declaration, not an expression
}

type_t addConst(type_t t)
{
    return const(t);    // `const(t)` only parses in type context
}
```

Before: std/traits.d

```
template AllImplicitConversionTargets(T)
{
    static if (is(T == bool))
        alias AllImplicitConversionTargets = AliasSeq!(byte, AllImplicitConversionTargets!byte);
    else static if (is(T == byte))
        alias AllImplicitConversionTargets = AliasSeq!(char, ubyte, short, AllImplicitConversionTargets!short);
    else static if (is(T == ubyte))
        alias AllImplicitConversionTargets = AliasSeq!(byte, char, short, AllImplicitConversionTargets!short);
    else static if (is(T == short))
        alias AllImplicitConversionTargets = AliasSeq!(ushort, wchar, int, AllImplicitConversionTargets!int);
    else static if (is(T == ushort))
        alias AllImplicitConversionTargets = AliasSeq!(short, wchar, dchar, AllImplicitConversionTargets!dchar);
    else static if (is(T == int))
        alias AllImplicitConversionTargets = AliasSeq!(dchar, uint, long, AllImplicitConversionTargets!long);
    else static if (is(T == uint))
        alias AllImplicitConversionTargets = AliasSeq!(dchar, int, long, AllImplicitConversionTargets!long);
    else static if (is(T == long))
        alias AllImplicitConversionTargets = AliasSeq!(ulong, CentTypeList, float, double, real);
    else static if (is(T == ulong))
        alias AllImplicitConversionTargets = AliasSeq!(long, CentTypeList, float, double, real);
    else static if (is(T == float))
        alias AllImplicitConversionTargets = AliasSeq!(double, real);
    else static if (is(T == double))
        alias AllImplicitConversionTargets = AliasSeq!(float, real);
    else static if (is(T == real))
        alias AllImplicitConversionTargets = AliasSeq!(float, double);
    else static if (is(T == char))
        alias AllImplicitConversionTargets = AliasSeq!(byte, ubyte, short, AllImplicitConversionTargets!short);
    else static if (is(T == wchar))
        alias AllImplicitConversionTargets = AliasSeq!(short, ushort, dchar, AllImplicitConversionTargets!dchar);
    else static if (is(T == dchar))
        alias AllImplicitConversionTargets = AliasSeq!(int, uint, long, AllImplicitConversionTargets!long);
    else static if (is(T == class))
        alias AllImplicitConversionTargets = staticMap!(ApplyLeft!(CopyConstness, T), TransitiveBaseTypeTuple!T);
    else static if (is(T == interface))
        alias AllImplicitConversionTargets = staticMap!(ApplyLeft!(CopyConstness, T), InterfacesTuple!T);
    else static if (isArray!T && !is(typeof(T.init[0]) == const))
    {
        static if (is(typeof(T.init[0]) == shared))
            alias AllImplicitConversionTargets = AliasSeq!(const(shared(Unqual!(typeof(T.init[0]))))[]);
        else
            alias AllImplicitConversionTargets = AliasSeq!(const(Unqual!(typeof(T.init[0]))))[];
    }
    else static if (is(T : void*) && !is(T == void*))
        alias AllImplicitConversionTargets = AliasSeq!(void*);
    else static if (is(cent) && is(T == cent))
        alias AllImplicitConversionTargets = AliasSeq!(UnsignedCentTypeList, float, double, real);
    else static if (is(ucent) && is(T == ucent))
        alias AllImplicitConversionTargets = AliasSeq!(SignedCentTypeList, float, double, real);
    else
        alias AllImplicitConversionTargets = AliasSeq!();
}
```

After: a table and a loop

```
enum type_t[][type_t] directTargets = [  
    bool : [byte],  
    byte : [char, ubyte, short],    ubyte : [byte, char, short],  
    short : [ushort, wchar, int],    ushort: [short, wchar, dchar],  
    int : [dchar, uint, long],    uint : [dchar, int, long],  
    long : [ulong, float, double, real],  
    ulong : [long, float, double, real],  
    float : [double, real],    double: [float, real],    real: [float, double],  
    char : [byte, ubyte, short],  
    wchar : [short, ushort, dchar], dchar : [int, uint, long],  
];  
  
type_t[] allImplicitConversionTargets(type_t t)  
{  
    type_t[] result;  
    foreach (target; directTargets.get(t, null))  
        if (!result.canFind(target))  
            result ~= target ~ allImplicitConversionTargets(target);  
    return result;  
}
```

What doesn't (always) work

Value vs. type conflation:

```
static_assert(3.sizeof == int.sizeof); // both are 4

type_t largerType(type_t a, type_t b)
{
    return a.sizeof > b.sizeof ? a : b; // sizeof of the type_t, or of the type?
}
```

alias parameters have the same problem:

```
enum staticArraySize(alias T, size_t length) = T.sizeof * length;
```

Takeaway

- **Reify** → **compute** → **dereify** is already how we manipulate types
- Today that round trip goes through `.mangleof` and `__traits(toType, ...)`
- `type_t` would make step 2 just... **code**
- Most of it parses already — the hard parts are `T*`, `const(T)` and `.sizeof`